Computing Technology Project A - Research Report

LLM-Assisted Review and Improvement of Sigma Detection Rules in a Detection-as-Code Pipeline

Ngo Sy Vuong
Department of Computer Science
Swinburne University of Technology
Hanoi, Vietnam
ngosyvuong15022006@gmail.com

Abstract— Detection Engineering in Security Operations Centres (SOCs) faces ongoing challenges in developing, reviewing, and maintaining effective detection rules, including false positives, inconsistent rule quality, and the reliance on manual security expertise. This paper investigates the implementation of a Large Language Model (LLM) within a Detection-as-Code (DaC) workflow to assist Cybersecurity Leads in reviewing and improving Sigma detection rules. The proposed approach integrates an LLM into a GitHub-based DaC pipeline, where submitted Sigma rules can be analysed for potential issues and receive suggested improvements before proceeding through automated validation and testing. Deterministic validation is performed using pySigma, while MITRE ATT&CK provides threat context and Atomic Red Team supports controlled behavioural testing. The LLM is designed as an advisory component rather than an autonomous decision-maker, maintaining human oversight throughout the detection development process. The research evaluates how LLM-generated reviews and suggestions can complement existing DaC practices while considering the reliability, limitations, and risks of AI-assisted detection engineering. The proposed approach aims to reduce repetitive manual review, improve the consistency of Sigma rule development, and provide a practical method for integrating AI assistance into a reproducible Detection-as-Code workflow.

Keywords— Large Language Models, LLM, Detection-as-Code, Sigma, Cybersecurity Detection, Detection Engineering, MITRE ATT&CK, GitHub Actions.

I. RESEARCH ON THE BUSINESS DOMAIN

A. Cybersecurity Detection and Detection Engineering

Cybersecurity detection is a core activity in Security Operations Centres (SOCs), where security data from endpoints, networks, servers and applications is analysed to identify potentially malicious behaviour. Detection rules convert known threat behaviours into machine-readable conditions that can generate alerts for investigation.

However, research shows that detection effectiveness depends not only on the number of rules but also on how they are developed and managed. Vermeer et al. [12] analysed approximately 130,000 network intrusion detection rules and found that only 0.5% of rules generated more than 80% of alerts and incidents, while 16% of investigated incidents were labelled false positives. These findings indicate that simply increasing the number of detection rules does not necessarily improve security outcomes. Instead, the effectiveness of a detection depends on factors such as its specificity, the alerts it produces, and how it performs in operational use.

Similarly, Vermeer [11] found that rule specificity, alert volume and false positives influenced how SOC professionals managed detection rules. This suggests that detection engineering is not a one-time process in which a rule is created and deployed unchanged. Rules may need to be reviewed and refined based on their operational outcomes, making continuous evaluation an important part of detection development. For this reason, approaches that can support the repeated review and improvement of detection rules may provide practical value to SOC teams.

B. Challenges in Detection Development

One major challenge is maintaining useful detections while reducing unnecessary alerts. Spathoulas and Katsikas [8] demonstrated that statistical post-processing could substantially reduce false positives in intrusion detection alerts, highlighting the operational cost of noisy security information. This indicates that detection development should consider not only whether a rule can generate an alert, but also whether the resulting alerts are sufficiently useful for security analysts.

Another challenge is determining whether a detection works beyond the specific example used during development. Teuwen et al. [9] studied 60 participants who produced 3,146 network detection rules and identified difficulties in generalising rules beyond the tests available to engineers. This suggests that a rule may appear effective against a known test case while still having limitations when applied to other behaviours. Therefore, detection development requires more than syntactic validation; rules also need systematic review and testing against relevant behaviours.

These challenges create an opportunity for AI-assisted detection engineering. An LLM could support the repeated review of existing rules by analysing their logic, identifying potential weaknesses and suggesting possible improvements. However, an LLM's ability to generate plausible recommendations does not demonstrate that those recommendations are correct or that the resulting detection is effective. AI-generated suggestions therefore require independent validation and behavioural testing before they can be relied upon in a security workflow. This creates a need to investigate not only whether an LLM can assist with Sigma rule review, but also how its recommendations can be integrated with deterministic validation, testing and human security judgement.

C. Detection-as-Code

Detection-as-Code (DaC) applies software-engineering practices such as version control, automated testing and CI/CD to detection development. Detection rules can be stored as code, reviewed through pull requests and automatically processed through a development pipeline.

DaC provides a suitable environment for AI-assisted detection development because rule review and automated checks can be implemented as repeatable stages. However, automation does not determine detection effectiveness. Research shows that detection quality depends on specificity, false positives, testing and operational feedback [9], [11], [12].

Therefore, this project treats the LLM as an advisory component. The LLM reviews Sigma rules and suggests improvements, while pySigma provides deterministic validation, controlled testing provides behavioural evidence, and the Cybersecurity Lead makes the final decision.

D. Detection Standards and Frameworks

The proposed workflow combines several technologies with complementary roles. Sigma provides the standard format for representing detection logic, making it suitable as the primary format for LLM-assisted rule review. MITRE ATT&CK provides threat context by linking detections to adversary tactics and techniques; however, ATT&CK mapping alone does not demonstrate real-world detection coverage [13]. Atomic Red Team provides controlled tests for relevant adversary behaviours.

These technologies therefore support different stages of the workflow: Sigma represents the rule, the LLM reviews and suggests improvements, ATT&CK provides threat context, and Atomic Red Team provides behavioural testing. This separation allows the LLM to assist with rule analysis without replacing deterministic validation, testing or human security judgement.

E. Literature Review and Research Gap

Previous research highlights that effective detection engineering depends on more than creating a large number of rules. Vermeer et al. [12] found that a small proportion of rules generated a large proportion of alerts and incidents, while Vermeer [11] showed that rule specificity, alert volume and false positives influenced SOC rule management. These findings indicate that detection quality must be considered in operational use rather than measured only by the number of rules created.

Testing is another limitation. Teuwen et al. [9] identified difficulties in generalising detection rules beyond the tests available during development. This suggests that a rule can perform well against a known example while remaining ineffective against other relevant behaviours. Therefore, rule review and behavioural testing should complement syntactic validation.

Recent research introduces LLMs as another approach to detection development. Cai et al. [2] demonstrated the feasibility of generating Sigma rules from cyber threat reports, while Bertiger et al. [1] highlighted the need for systematic evaluation of LLM-generated detection rules. Taken together, these studies suggest that LLMs can reduce part of the manual effort involved in detection development, but model-generated output should not be treated as evidence of detection effectiveness.

The literature therefore points to a need for combining AI-assisted analysis with independent validation and testing. This project focuses on the practical implementation of that approach by integrating an LLM to review and improve existing Sigma rules within a Detection-as-Code workflow. Unlike treating the LLM as an autonomous rule generator, the proposed approach uses the model as an advisory component whose recommendations are checked using deterministic validation, behavioural testing and human security judgement.

F. Research Methodology

This research adopts a design-oriented and experimental methodology to investigate how a Large Language Model (LLM) can assist in reviewing and improving Sigma detection rules. The research consists of three stages.

First, a literature review examines research on detection engineering, LLM-based detection rules, Sigma, automated validation and human-AI collaboration. The findings are synthesised to identify limitations in existing approaches and define requirements for the proposed LLM-assisted workflow.

Second, a prototype implementation integrates an LLM into a GitHub-based Detection-as-Code workflow. Selected Sigma rules are submitted to the LLM together with defined review criteria, and the model produces structured feedback and suggested improvements. The original and modified rules are then checked using deterministic tools such as pySigma.

Third, the approach is experimentally evaluated using selected Sigma rules and controlled testing scenarios. The

original and LLM-modified rules are evaluated under the same validation and testing conditions. Evaluation considers technical validity, correctness and relevance of recommendations, consistency of LLM outputs, and behavioural performance of modified rules where testing is applicable. Human review is also retained to assess whether the recommendations are practically useful and appropriate for detection development.

The methodology therefore combines literature analysis, prototype implementation and empirical evaluation to determine whether LLM assistance can improve Sigma rule review while maintaining deterministic safeguards and human security judgement.

G. Proposed Solution and Objectives

The proposed solution is a GitHub-based Detection-as-Code workflow that incorporates an LLM into the detection-rule review process. When a Sigma rule is submitted or modified through a pull request, the workflow can provide the rule to the LLM together with relevant context and review criteria. The LLM analyses the rule and produces feedback, such as potential weaknesses, explanations and suggested improvements.

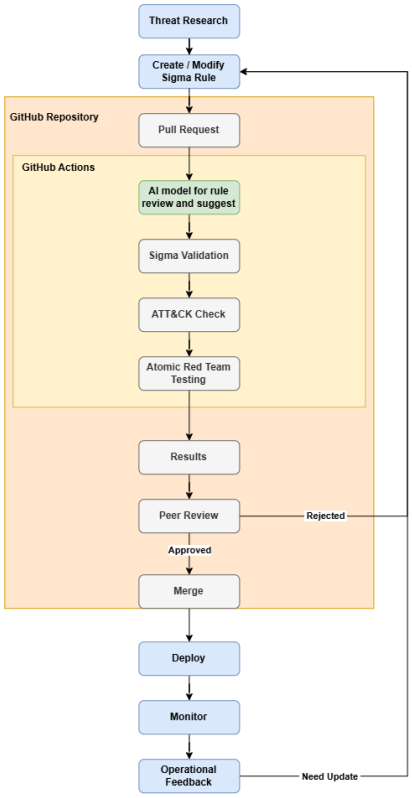


Figure 1 – Proposed Workflow (v0.3)

The AI review is followed by existing deterministic and practical validation stages. pySigma can be used to validate the technical structure of the Sigma rule, while MITRE ATT&CK provides threat context and Atomic Red Team can support controlled behavioural testing. The Cybersecurity Lead then reviews the AI suggestions and validation results before deciding whether the rule should be accepted.

The main objective is therefore to investigate how an LLM can be implemented to assist with the review and improvement of Sigma detection rules within a Detection-as-Code pipeline. Supporting objectives are to reduce repetitive manual review, improve the consistency of rule development, maintain traceability through Git and pull requests, and evaluate the reliability and limitations of LLM-generated recommendations.

II. Research on End Users

The proposed Detection-as-Code system has two primary end users: the Cybersecurity Lead / Detection Engineer and the Software / DevOps Engineer. The Detection Engineer focuses on the security quality of detection rules, while the Software / DevOps Engineer maintains the automation infrastructure. The LLM primarily supports the Detection Engineer by providing rule review and improvement suggestions.

A. Cybersecurity Lead / Detection Engineer

The Cybersecurity Lead is responsible for researching threats and developing and reviewing detection rules. Teuwen et al. [9] identified difficulties in generalising detection rules beyond available tests, indicating the need for reliable review, testing and feedback.

The proposed system supports these needs by using an LLM to review Sigma rules and suggest potential improvements, while pySigma, MITRE ATT&CK and controlled testing provide additional validation and context. However, LLM suggestions may not always be correct. Research on human-automation interaction indicates that automation can reduce workload but may also introduce automation bias (Tilbury and Flowerday [10]). Therefore, the Detection Engineer remains responsible for evaluating AI suggestions and making the final deployment decision.

B. Software / DevOps Engineer

The Software / DevOps Engineer maintains the pipeline connecting GitHub, the LLM, validation tools and testing frameworks. GitHub Actions can trigger the LLM review when a Sigma rule is submitted or modified through a pull request. The engineer is responsible for reliable execution, model/API integration and recording pipeline results, but does not make the final security decision.

The two users therefore have complementary roles: the Cybersecurity Lead provides security expertise and human judgement, while the Software / DevOps Engineer provides the infrastructure for consistent AI-assisted automation.

III. Research on the Solution Domain

The proposed system integrates existing technologies into a Detection-as-Code workflow rather than developing new cybersecurity technologies. The main components are Sigma/pySigma, the LLM, MITRE ATT&CK, Atomic Red Team, GitHub and GitHub Actions.

A. Sigma Rules

Sigma provides a vendor-independent format for representing detection logic, making it suitable for storing and reviewing detection rules as code. pySigma can parse, validate and convert Sigma rules into target-specific queries [7]. However, syntactic validity does not demonstrate detection effectiveness. Therefore, pySigma is used for deterministic validation, while the LLM provides additional semantic review and improvement suggestions.

C. LLM Implementation for Sigma Rules review

The LLM is the main AI component of the proposed workflow. Rather than generating detections autonomously, it reviews an existing Sigma rule and identifies potential weaknesses, explains the issues and suggests possible improvements. The input can include the Sigma rule, defined review criteria and relevant ATT&CK context, while the output should provide structured feedback and proposed changes.

The implementation should consider the trade-offs between a hosted LLM API and a locally deployed model. A hosted API can simplify integration and provide access to capable models, while a local model can provide greater control over security-sensitive data. The selection should therefore consider model capability, privacy, cost and implementation complexity. The LLM output is advisory and must be checked by the subsequent validation and testing stages.

The LLM review can be triggered by a GitHub Actions workflow when a Sigma rule is submitted or modified through a pull request. The workflow can provide the Sigma YAML, review criteria and relevant ATT&CK context to the model through an API or local inference service. The model can return structured feedback containing identified issues, explanations and proposed changes. These outputs can then be stored as a pull-request comment or workflow artifact before the rule proceeds to pySigma validation and controlled testing.

The choice between a hosted API and a locally deployed model also presents a practical trade-off. A hosted API reduces infrastructure complexity and can provide access to stronger general-purpose models, but introduces external data-processing and usage costs. A local model provides greater control over sensitive rule and log data but requires additional infrastructure and model maintenance. Therefore, the appropriate approach should be selected based on the project's requirements for model capability, privacy, cost and implementation complexity.

D. MITRE ATT&CK

MITRE ATT&CK provides a structured representation of adversary tactics and techniques and can provide threat context for Sigma rules [3]. However, ATT&CK mapping does not prove detection effectiveness. Therefore, it is used as contextual information alongside LLM review and functional testing.

E. Atomic Red Team

Atomic Red Team provides controlled tests corresponding to ATT&CK techniques [5]. These tests can generate relevant behaviours and help determine whether a detection responds as expected. Testing should be performed only in an authorised and isolated environment, with results retained as evidence.

D. GitHub and GitHub Actions

GitHub provides version control, pull requests and change history, while GitHub Actions provides the CI/CD mechanism for automatically processing modified Sigma rules. The pipeline can trigger LLM review, validation and testing when a rule is submitted or modified. Python can support rule processing, ATT&CK handling and integration with the required tools.

F. Critical Analysis of the Proposed Technical Solution

The main strength of the proposed architecture is the separation of AI assistance from deterministic and behavioural safeguards. The LLM can analyse rule logic and provide recommendations that are difficult to obtain through purely syntactic validation, while pySigma can verify whether the rule is technically valid and Atomic Red Team can provide behavioural evidence. However, these components have different limitations: LLM output may be incorrect or inconsistent, pySigma cannot establish detection effectiveness, and controlled tests cannot represent every possible real-world behaviour.

This makes a fully autonomous LLM-based workflow unsuitable for the proposed system. A human reviewer is therefore retained as the final decision-maker, while the LLM is used to reduce repetitive review effort. The architecture consequently favours complementary automation rather than replacement, balancing the potential efficiency of AI assistance against reliability, testing and human-oversight requirements.

IV. Cost Analysis

The proposed system mainly uses existing development resources, including GitHub, GitHub Actions, Python, Sigma/pySigma and a controlled testing environment. The main additional cost is LLM usage if a hosted API is selected, as costs will depend on model capability and the number of rules reviewed. A locally hosted model could reduce external API costs and provide greater control over security-sensitive data, but would introduce additional hardware and maintenance requirements.

The project also has a practical resource gap in detection engineering and security infrastructure. The Cybersecurity Lead focuses on threat research and Sigma development, while the Software / DevOps Engineer focuses on CI/CD and LLM integration. Sharing knowledge and reviewing the workflow across both roles can reduce reliance on specialised external resources.

V. Ethical Considerations

The proposed Detection-as-Code system involves security logs, controlled cybersecurity testing and AI-assisted detection development. The main ethical considerations are privacy and data protection, authorised security testing, responsible use of AI, and access control.

The Australian Privacy Principles (APPs), under the Privacy Act 1988 (Cth), provide a relevant framework for handling personal information [4]. APP 3 requires personal information collection to be reasonably necessary, while APP 11 requires reasonable protection against misuse, loss and unauthorised access [4]. Therefore, the project will minimise the use of real personal data and use synthetic or de-identified security logs where possible.

Security testing must also be authorised. Atomic Red Team tests can simulate adversary behaviours, so testing will be restricted to an isolated environment controlled by the project team [5]. Production and third-party systems will not be tested.

Responsible use of the LLM is particularly important because AI-generated recommendations may contain incorrect, incomplete or inappropriate detection logic. Automatically accepting such recommendations could introduce ineffective or misleading security controls. Therefore, the LLM will operate as an advisory component rather than an autonomous decision-maker. The Cybersecurity Lead will review LLM recommendations together with validation and testing results before accepting a detection.

Finally, access to the GitHub repository, security rules and testing evidence will be restricted to authorised team members. Version control will maintain traceability of rule changes, while unnecessary sensitive information will not be stored.

VI. Future Directions

Future development could extend the proposed workflow in three areas.

First, the LLM component could be expanded beyond reviewing and improving Sigma rules to support additional AI-assisted tasks, such as rule generation, rule explanation, ATT&CK mapping assistance and detection coverage analysis. This direction is supported by research demonstrating the feasibility of LLM-generated detections while also highlighting the need for systematic evaluation of their quality (Cai [2]; Bertiger [1]).

Second, the workflow could support additional SIEM query languages and security platforms through more pySigma backends. This would allow the same Sigma-based detection and AI-assisted review process to be translated and deployed across multiple SIEM environments rather than being limited to a single platform.

Third, detection testing and operational feedback could be expanded. Testing could cover more ATT&CK techniques and multiple scenarios, while operational information such as false-positive rates and alert outcomes could be incorporated to support continuous rule improvement. This addresses the difficulty of generalising detections beyond available tests and the importance of operational feedback in SOC rule management (Teuwen et al. [9]; Vermeer [11]).

References

[1] A. Bertiger, "Evaluating LLM Generated Detection Rules in Cybersecurity," in *Proc. Mach. Learn. Res.*, vol. 299, 2025. [Online]. Available: https://proceedings.mlr.press/v299/bertiger25a.html

[2] Y. Cai, "From Texts to Rules: Generating Sigma Rules with Large Language Models from Cyber Threat Reports," in *Proc. USENIX Secur. Symp.*, 2026. [Online]. Available: https://www.usenix.org/conference/usenixsecurity26/presentation/cai

[3] MITRE, "MITRE ATT&CK," *MITRE ATT&CK*. [Online]. Available: https://attack.mitre.org/techniques/

[4] Office of the Australian Information Commissioner, "Australian Privacy Principles guidelines," 2026. [Online]. Available: https://www.oaic.gov.au/privacy/australian-privacy-principles/australian-privacy-principles-guidelines

[5] Red Canary, "Atomic Red Team," *Atomic Red Team*. [Online]. Available: https://www.atomicredteam.io/docs/atomic-red-team

[6] SigmaHQ, "Backends," *SigmaHQ*. [Online]. Available: https://sigmahq.io/docs/digging-deeper/backends

[7] SigmaHQ, "pySigma Documentation," *SigmaHQ*. [Online]. Available: https://sigmahq-pysigma.readthedocs.io/en/latest/getting_started.html

[8] G. P. Spathoulas and S. K. Katsikas, "Reducing false positives in intrusion detection systems," *Comput. Secur.*, vol. 29, no. 1, 2010, doi: 10.1016/j.cose.2009.07.008.

[9] K. T. W. Teuwen, E. Zambon, and L. Allodi, "How It's Made: Uncovering Detection Engineering Processes for Network Intrusion Detection Rules," 2026, doi: 10.1145/3846375.3849098.

[10] J. Tilbury and S. Flowerday, "Humans and Automation: Augmenting Security Operation Centers," *J. Cybersecur. Priv.*, vol. 4, no. 3, pp. 388–409, 2024, doi: 10.3390/jcp4030020.

[11] M. Vermeer, "Alert Alchemy: SOC Workflows and Decisions in the Management of Network Intrusion Detection Rules," 2023, doi: 10.1145/3576915.3616581.

[12] M. Vermeer, M. Van Eeten, and C. Gañán, "Ruling the Rules: Quantifying the Evolution of Rulesets, Alerts and Incidents in Network Intrusion Detection," 2022, doi: 10.1145/3488932.3517412.

[13] A. Virkud, "How Does Endpoint Detection Use the MITRE ATT&CK Framework?" in *Proc. USENIX Secur. Symp.*, 2024. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/virkud

[14] C. Weir, A. Rashid, and J. Noble, "Challenging software developers: Dialectic as a foundation for security assurance techniques," *J. Cybersecur.*, 2020, doi: 10.1093/cybsec/tyaa007.